

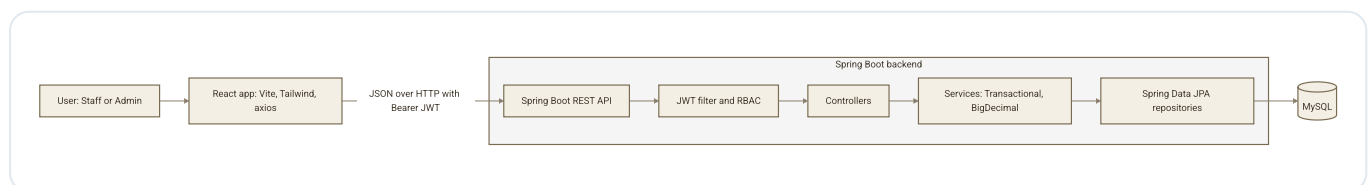
ManHourBooking: how the system works

Flow charts for the React frontend, the Spring Boot backend, and how the two connect. Based on the project spec and the frontend built in this chat.

1. [Architecture](#)
2. [Connecting to Spring Boot](#)
3. [Login and JWT flow](#)
4. [Staff workflow](#)
5. [Booking calculation](#)
6. [Admin workflow](#)
7. [Data model](#)
8. [API and backend layout](#)
9. [Rules and checklist](#)

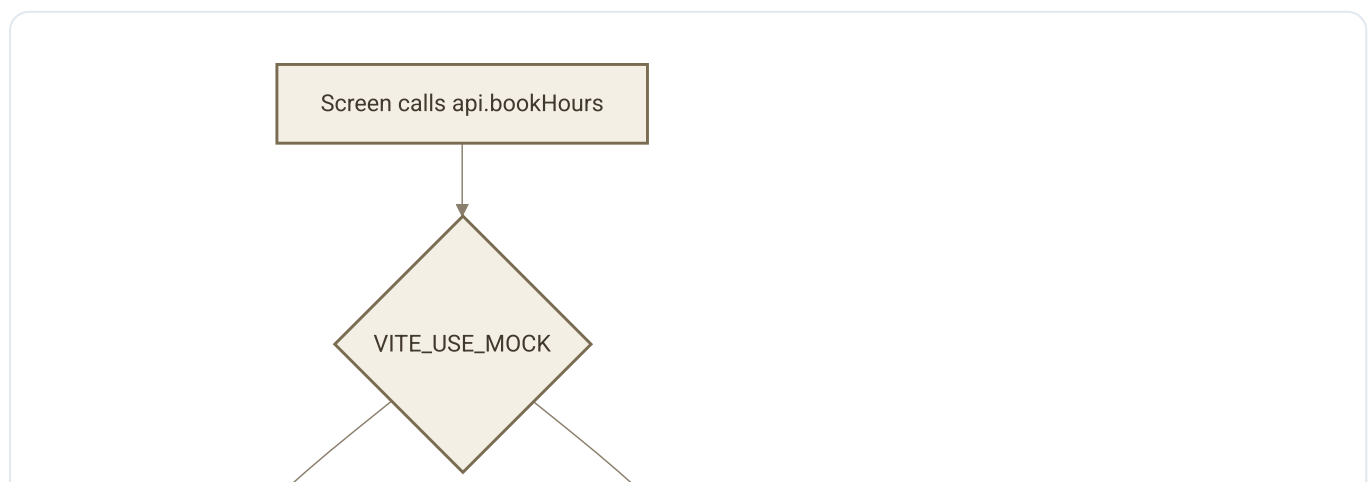
1. Architecture

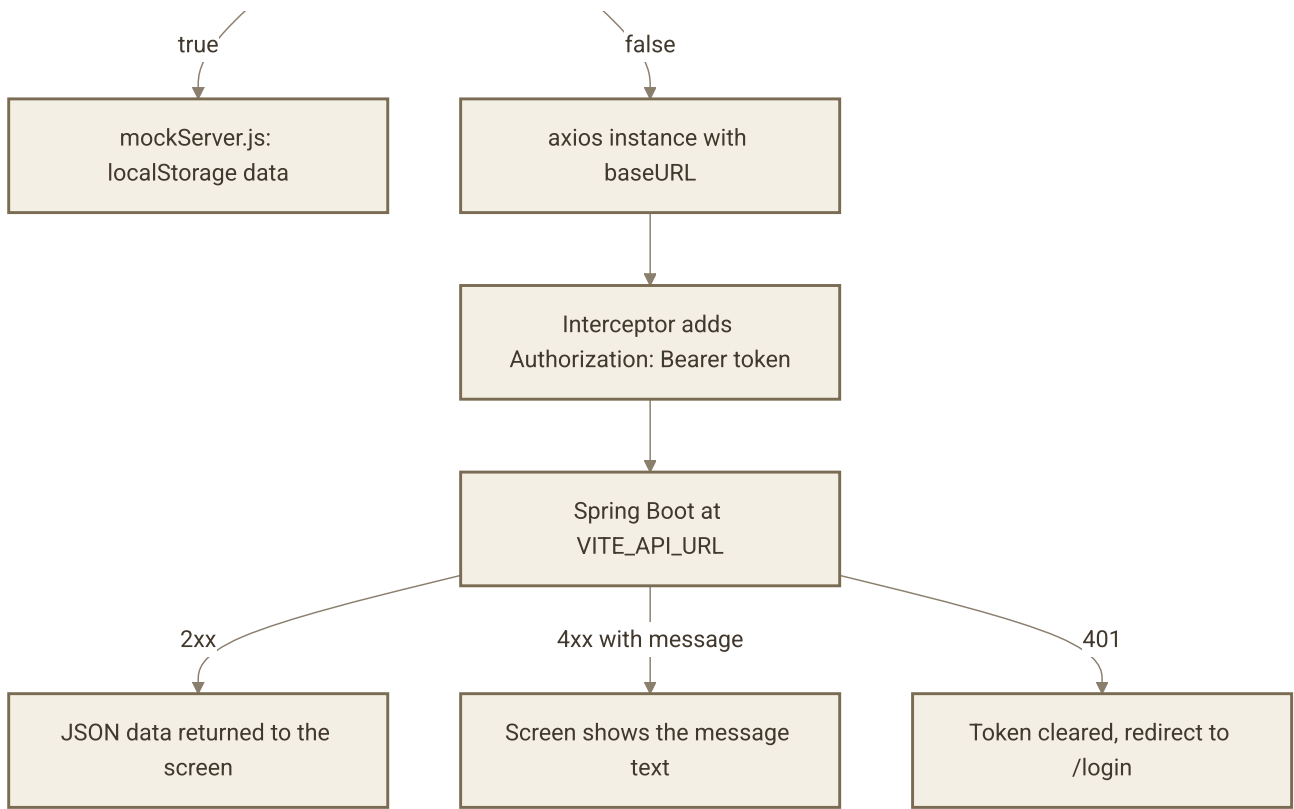
The browser runs the React app. It talks to Spring Boot over JSON REST calls, and Spring Boot talks to MySQL. Every request after login carries a JWT, and Spring Security checks the role before the controller runs.



2. Connecting the frontend to Spring Boot

The frontend has a single switch. While `VITE_USE MOCK=true` it uses a browser mock, so you can build screens before the backend exists. Changing it to `false` sends the same calls to Spring Boot with no UI changes.





Step by step

1. Create the backend with Spring Initializr: Java 17+, Maven, dependencies Spring Web, Spring Data JPA, Spring Security, MySQL Driver, Validation, plus a JWT library such as jjwt.
2. Run it on port 8080 and create the MySQL schema (see section 7).
3. In the frontend, copy `.env.example` to `.env` and set:

```
VITE_USE MOCK=false  
VITE_API_URL=http://localhost:8080
```

4. Allow the Vite dev origin with CORS in Spring Boot, otherwise the browser blocks every call:

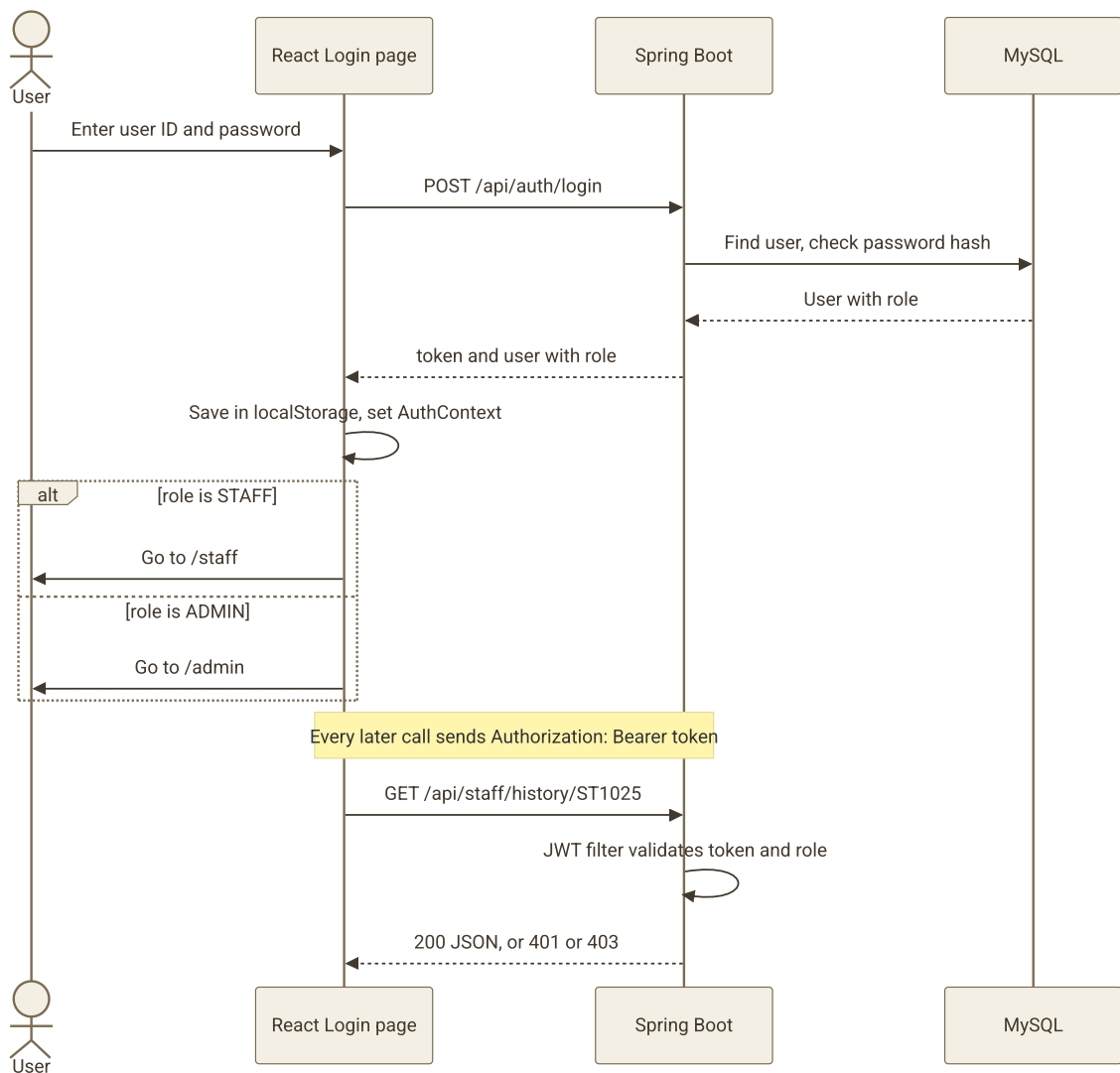
```
@Configuration  
public class CorsConfig {  
    @Bean  
    public CorsConfigurationSource corsConfigurationSource() {  
        CorsConfiguration c = new CorsConfiguration();  
        c.setAllowedOrigins(List.of("http://localhost:5173"));  
        c.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));  
        c.setAllowedHeaders(List.of("Authorization", "Content-Type"));  
        UrlBasedCorsConfigurationSource s = new UrlBasedCorsConfigurationSource();  
        s.registerCorsConfiguration("/**", c);  
        return s;  
    }  
}
```

}

5. Return errors as `{ "message": "..."}` with a 4xx status. The frontend shows that text directly. A `@RestControllerAdvice` class does this in one place.
6. Restart `npm run dev` (Vite reads `.env` only at start), then log in.

Alternative to CORS: add a Vite proxy for `/api` to `http://localhost:8080` and set `VITE_API_URL` to an empty string. Then the browser sees one origin.

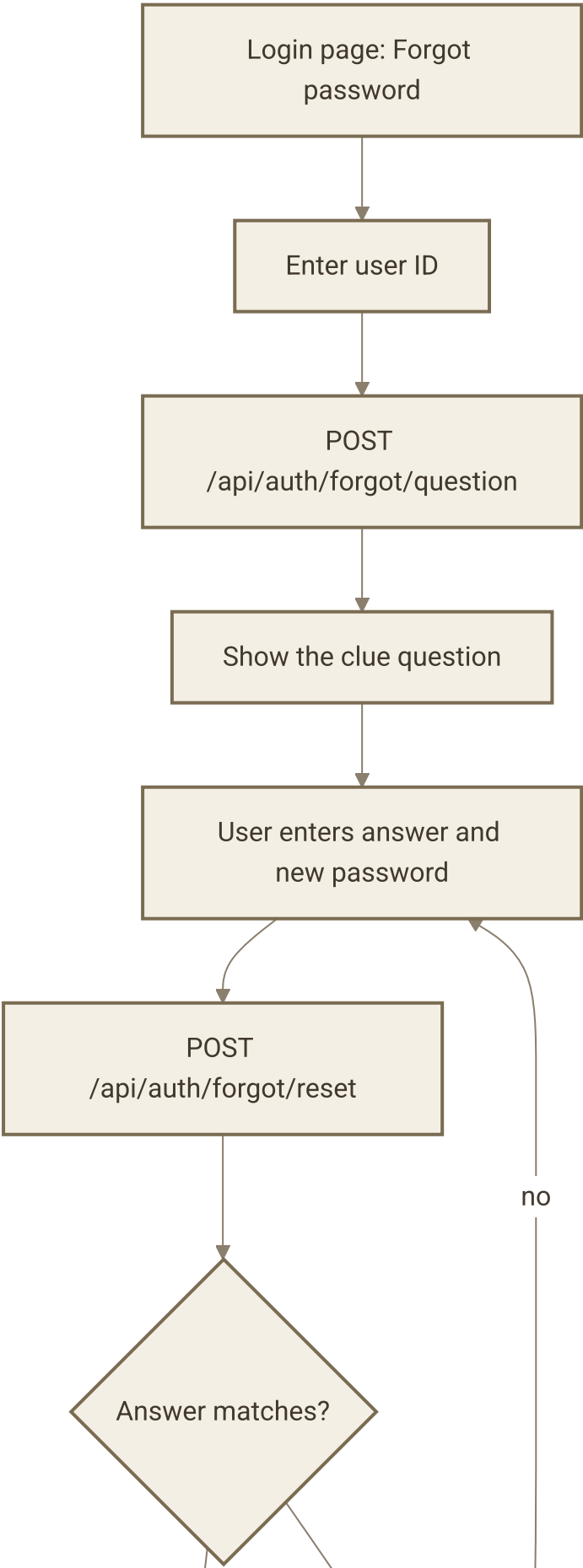
3. Login and JWT flow

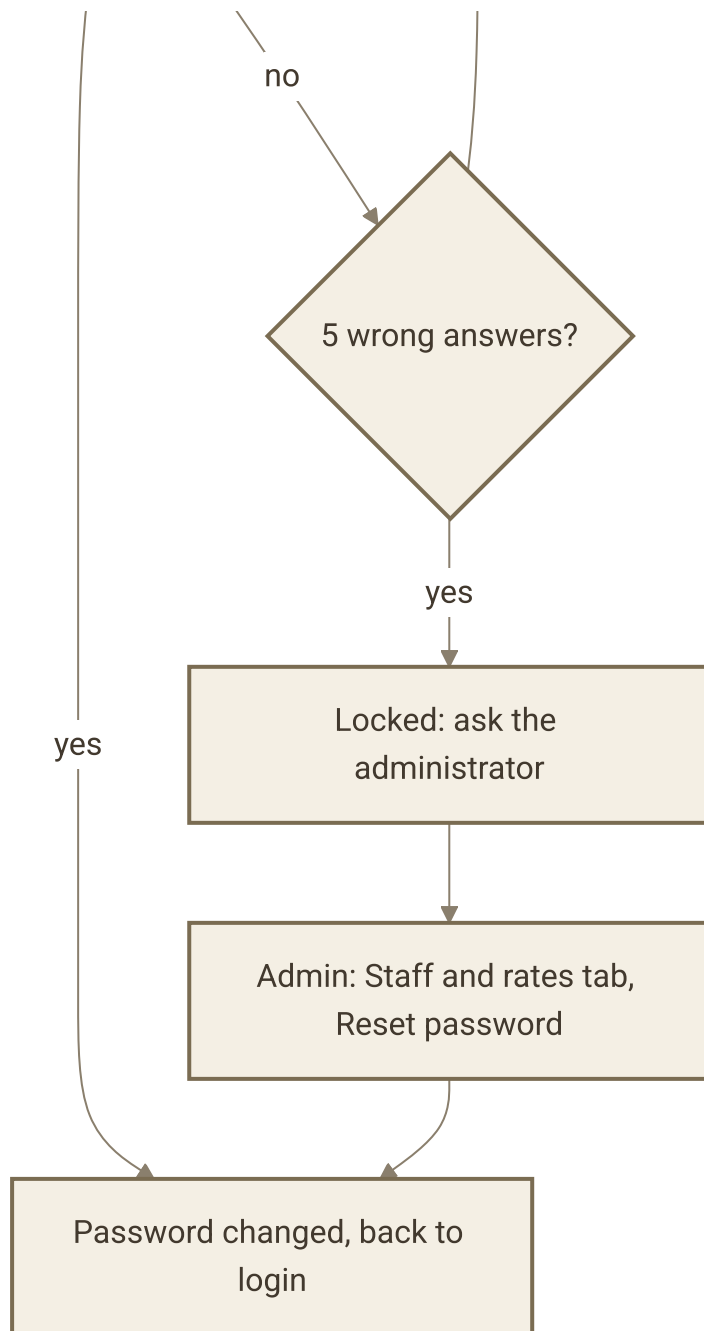


Sign-up uses `POST /api/auth/signup`. It always creates a STAFF user. The password and the clue answer are both stored hashed (BCrypt). Admin accounts are inserted directly by the system owner.

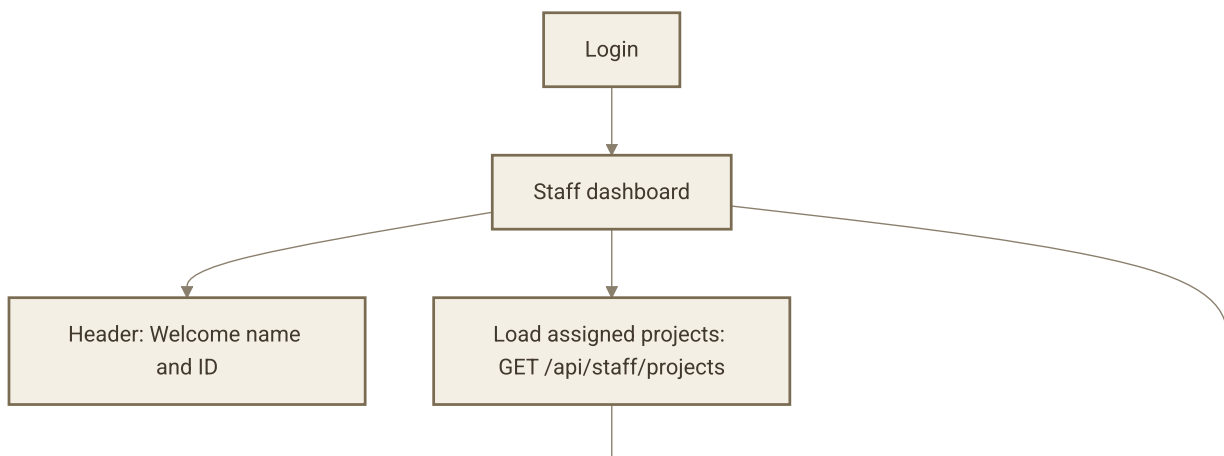
Forgot password (offline, with a clue)

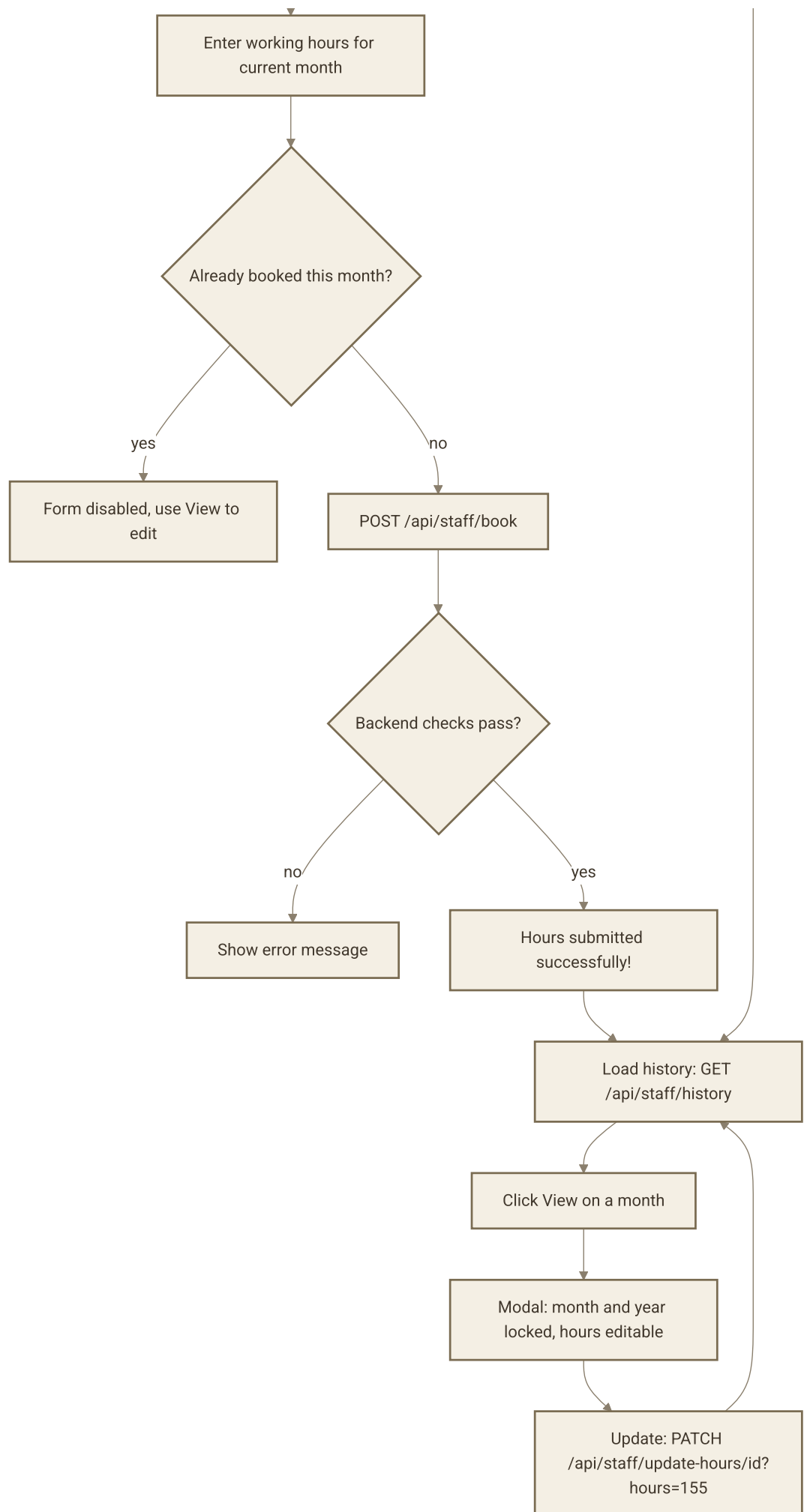
The system is offline, so there is no email. At sign-up each user picks a clue question and gives an answer. To recover access they answer that clue.





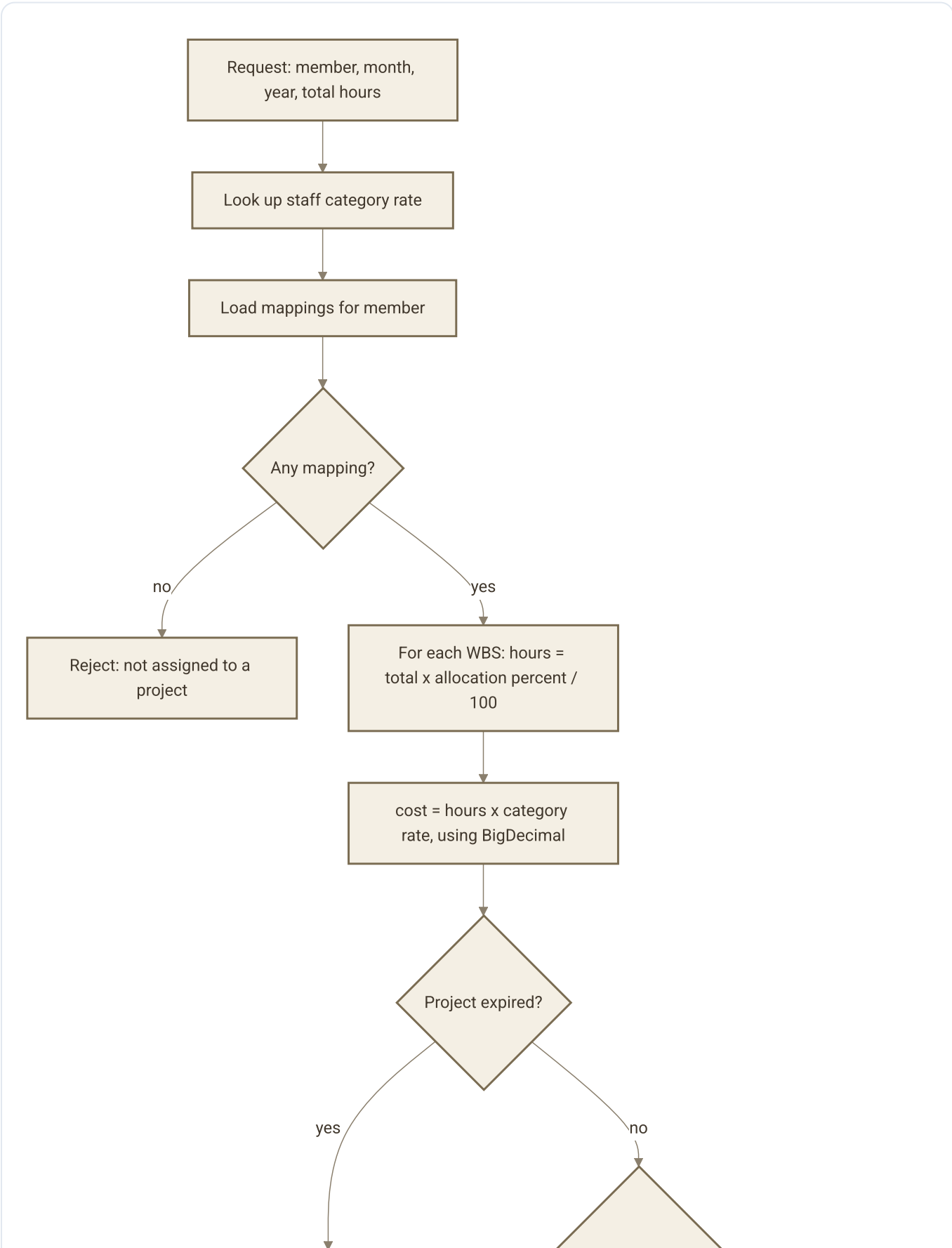
4. Staff workflow

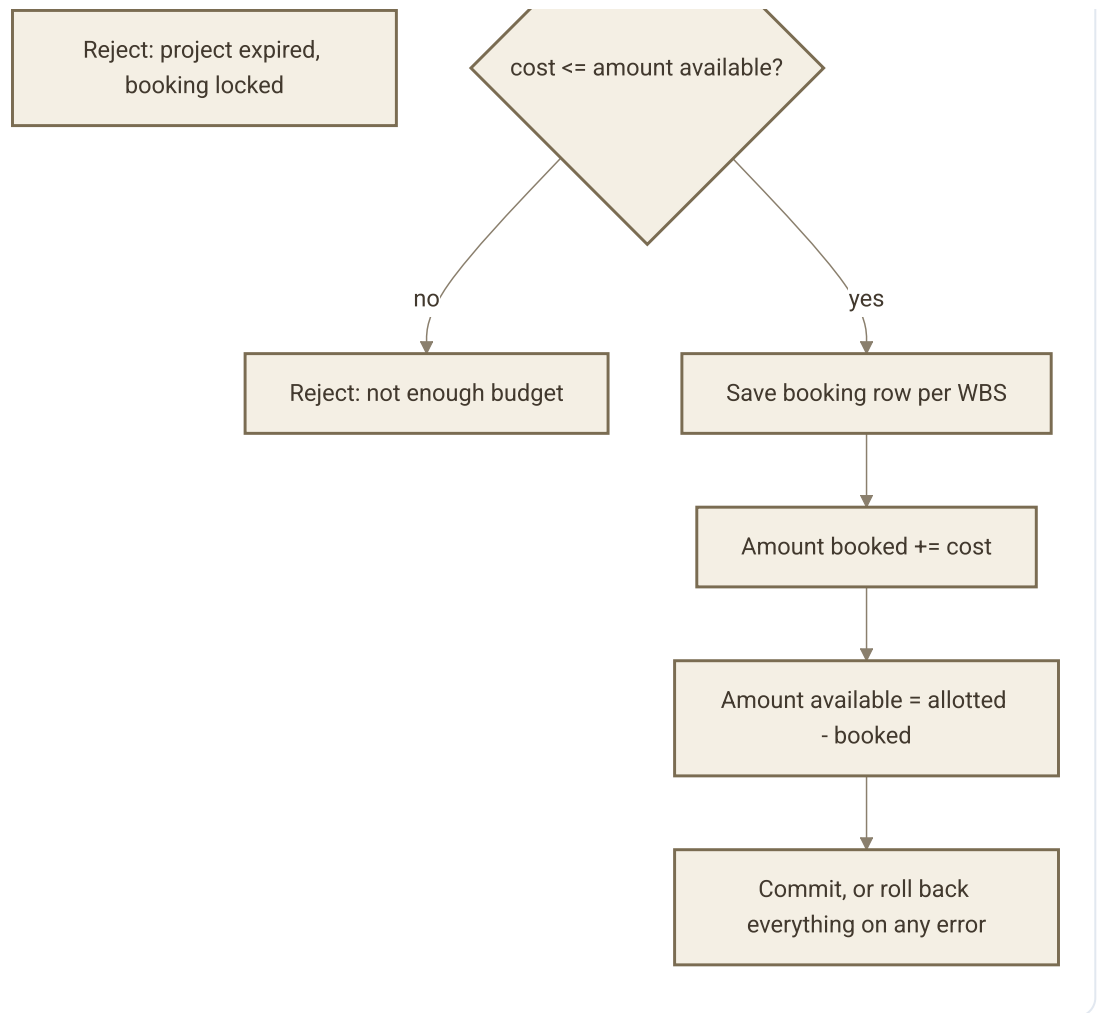




5. Booking calculation and budget

A staff member enters one total for the month. The backend splits it across the projects they are mapped to, prices each part, and updates each project budget in one transaction.

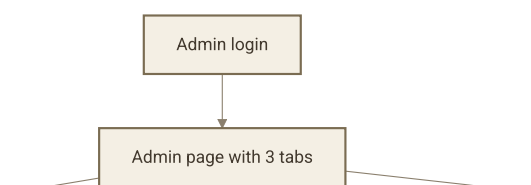


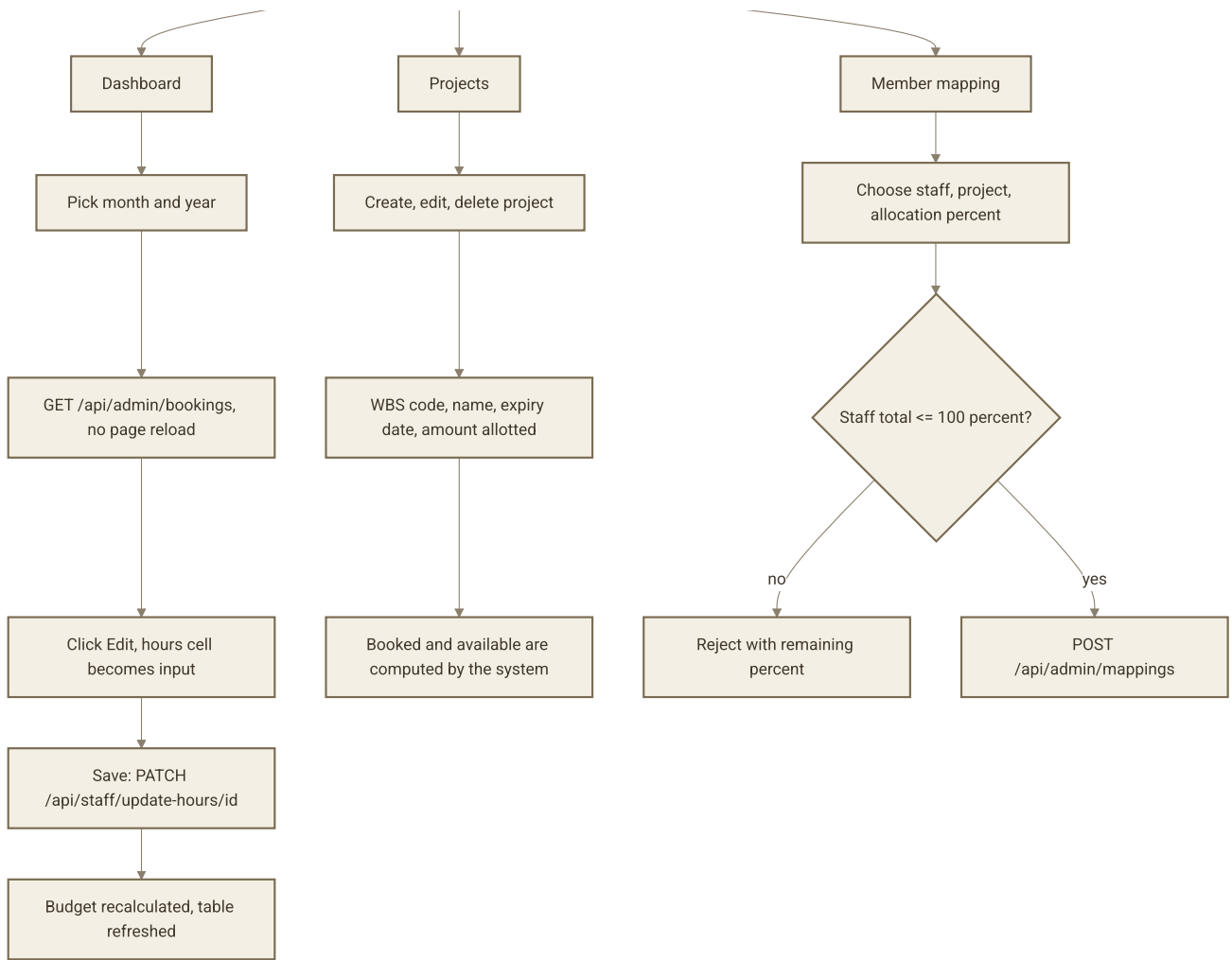


Example	Value
Staff total and rate	160 hours, staff in a category priced at 500 per hour
Allocation	60% on WBS-1001, 40% on WBS-1002
WBS-1001	96 h x 500 = 48,000
WBS-1002	64 h x 500 = 32,000
Invariant	Allotted = Available + Booked, for every project, always

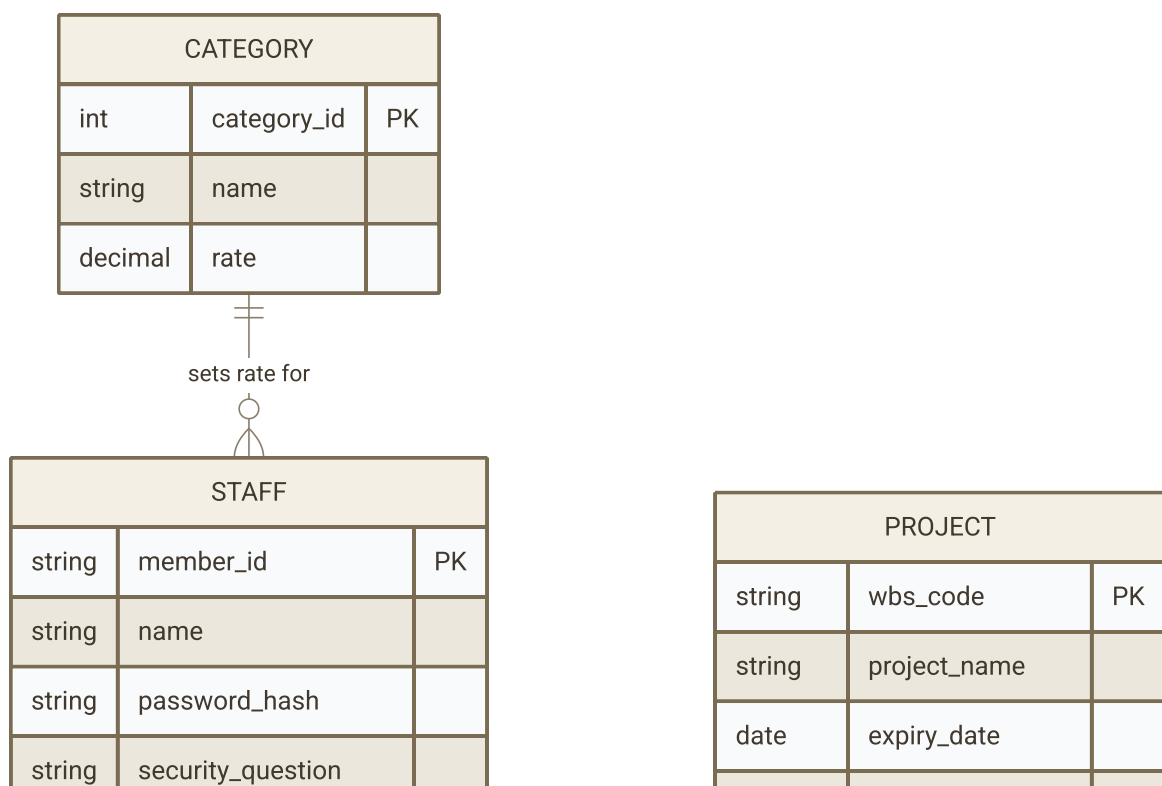
The staff edit (160 to 155) re-splits the new total across the month's rows and applies only the cost difference to each budget. An admin edit changes a single booking row the same way.

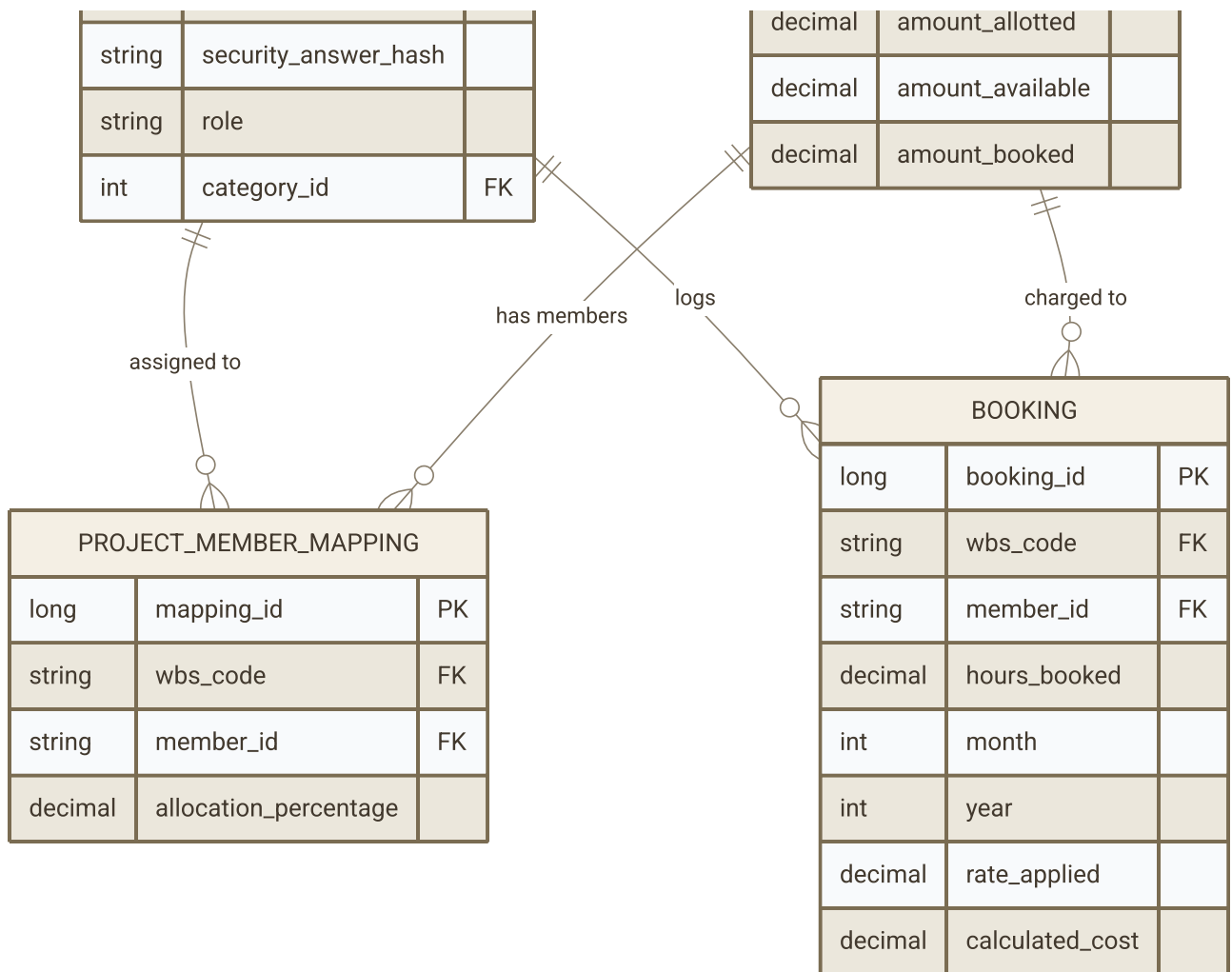
6. Admin workflow





7. Data model





The spec's Staff table has only `member_id` and `name` . Login needs a password hash and a role, and pricing needs a category, so add those columns (or a separate users table). The Category table holds the 5 rate categories. Add a unique key on (`member_id` , `wbs_code` , `month` , `year`) in Booking to stop double bookings.

8. API and backend layout

Method and path	Role	Purpose
POST /api/auth/login, /signup	Public	Authenticate, return token and user
POST /api/auth/forgot/question, /forgot/reset	Public	Offline password recovery with the clue question
GET /api/staff/projects/{memberId}	Staff	Only mapped projects (FR-1.1)
GET /api/staff/history/{memberId}	Staff	One row per month
POST /api/staff/book	Staff	Book month hours, split by allocation
PATCH /api/staff/update-hours/{id}?hours=	Staff,	Correct hours, recalculate budget

	Admin	
GET /api/admin/bookings?month=&year=	Admin	Dashboard data
GET, POST, PUT, DELETE /api/admin/projects	Admin	Project CRUD
GET /api/admin/staff, PUT /api/admin/staff/{id}/category	Admin	Staff with allocation and category; set category
GET, PUT /api/admin/categories	Admin	The 5 rate categories
PUT /api/admin/staff/{id}/password	Admin	Reset a staff password
GET, POST, DELETE /api/admin/mappings	Admin	Member to project mapping

The full request and response objects are in `API CONTRACT` and inside the frontend in